

CS 35L Software Construction

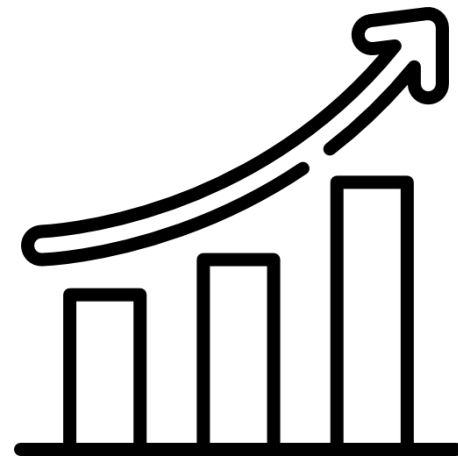
L17 – Code Comprehension & API Design

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department



Why Comprehension Skills Matter

- Developers spend **58% of their time** on **understanding code**
 - A speedup in code understanding has a **large impact** on productivity
- With AI writing more and more code, developers might spend even more time on reading code in the near future



Code Comprehension Approaches

“Bottom-Up” (Novices)

- Reading code **line by line**
- **grouping** statements into logical "chunks" of higher abstraction
- Takes **more time** because each statement needs to be analyzed

“Top-Down” (Experts)

- Driven by a **hypothesis** of what the code does
- Looking for “**beacons**” (**key lines of code** that **reveal the core purpose** of a function or test the hypothesis)
- First getting the **big picture**, then **looking for details**

Move from here to there



Tips on How to Transition from Bottom-Up Towards Top-Down Code Reading

Build Domain Knowledge

The top-down approach is only possible when you are **familiar with the application domain**.

Without domain knowledge, programmers lack the necessary schemas to form expectations and are forced to fall back on bottom-up chunking.

Tip: Before looking at the source code, invest time in **reading requirements, architectural documents, and high-level system summaries**.

Understanding *what* the software is supposed to do in the real world provides the foundation for your top-down expectations.

Tips on How to Transition from Bottom-Up Towards Top-Down Code Reading

Practice Hypothesis-Driven Reading

Instead of tracing execution flows immediately, **practice generating an initial, general hypothesis about what a program or module does** based on its name, context, or documentation.

Tip: Once you have a primary hypothesis (e.g., “This program produces invoices”), subdivide it into finer subsidiary hypotheses (e.g., “It must have a data retrieval method and a formatting method”) before looking at the code. You then only read the code to validate or reject these specific expectations.

Tips on How to Transition from Bottom-Up Towards Top-Down Code Reading

Hunt for “Beacons”

A top-down reader **does not read every line**; they scan for “beacons”. Beacons are stereotypical segments of code, recognizable variable names, or distinct structures that reliably indicate the presence of a specific operation or algorithm. At its core, a beacon is a **recognizable, familiar point in the source code that serves as a mental shortcut for the programmer**. They are defined as **“signs standing close to human thinking that may give a hint for the programmer about the purpose of the examined code”**

Tip: Train yourself to **scan files for recognizable patterns**, method signatures, and comments. When you spot a beacon, use it to instantly confirm your hypothesis without needing to trace the granular logic.

Beacons – The Structural Signs Guiding Your Through the Code

Identifier Names Are Beacons

The most frequent and most critical beacons are the **names developers assign to variables, functions, and classes**. Comprehension often depends **domain information** carried by identifier names.

Tip: First look for the names of important classes, functions, and variables. If you don't know their meaning (e.g., due to lack of domain knowledge) look up their meaning first!

Identifier Names — Code Examples

Spot the Domain in the Names – What do the functions do?

Python

```
def calculate_apr(  
    principal,  
    interest_rate,  
    term_months):  
    monthly = interest_rate/12  
    n = term_months  
    return (  
        (1+monthly)**n - 1  
    ) * 100
```

React (JSX)

```
function CartCheckout({  
    items, taxRate,  
    onPlaceOrder }) {  
    const subtotal = items  
        .reduce((s,i) =>  
            s + i.price, 0);  
    return <button onClick=  
        {onPlaceOrder}>  
        Pay {subtotal*taxRate}  
    </button>; }
```

Node.js (Express)

```
function authenticate(  
    req, res, next) {  
    const token = req  
        .headers.authorization;  
    if (!verifyJwt(token))  
        return res.status(401)  
            .send('Unauthorized');  
    next();  
}
```

Exercise: What domain does each snippet operate on? Which identifiers told you?

Beacons – The Structural Signs Guiding Your Through the Code

Identifier Names Are Beacons

The most frequent and most critical beacons are the **names developers assign to variables, functions, and classes**. Comprehension often depends **domain information** carried by identifier names.

Tip: First look for the names of important classes, functions, and variables. If you don't know their meaning (e.g., due to lack of domain knowledge) look up their meaning first!

Think-Pair-Share: What are other beacons besides identifier names?

Beacons – The Structural Signs Guiding Your Through the Code

Code Structures Are Beacons

Physical **layout** of these statements—often referred to as *text-structure knowledge*—serves as a visual beacon. Loops help you identify structural patterns. Guard clauses help you identify early exit paths.

Tip: Look for the **algorithms** used to solve the problem.

Code Structures — Code Examples

Recognize the Algorithm by Its Shape

Python

```
def max_depth(node):  
    # guard clause  
    if node is None:  
        return 0  
    # recursive shape  
    return 1 + max(  
        max_depth(node.left),  
        max_depth(node.right)  
    )
```

React (JSX)

```
useEffect(() => {  
    let attempts = 0;  
    // retry loop  
    while (attempts < 3) {  
        try {  
            return fetchData();  
        } catch {  
            attempts++;  
        }  
    }  
}, [url]);
```

Node.js

```
app.post('/orders', (req,res) => {  
    // guard  
    if (!req.user)  
        return res.status(401).end();  
    // map/reduce shape  
    const total = req.body.items  
        .map(i => i.price * i.qty)  
        .reduce((a,b) => a+b, 0);  
    res.json({ total });  
});
```

Exercise: Predict — what does each snippet compute? Which structural shape (loop, recursion, guard) gave it away?

Beacons – The Structural Signs Guiding Your Through the Code

Tests Are Beacons

When reading unfamiliar code, a developer's primary challenge is **deducing the original author's intent**. Tests act as explicit beacons that illuminate this intent by providing **an executable, unambiguous specification** of how the production code *should* work

Tip: When reviewing code, look at the tests first. Verify that the tests pass on the current code before spending too much time reading them

Tests — Code Examples

Tests Spell Out the Spec

Python (pytest)

```
def test_no_discount():
    assert apply(100, 0) == 100

def test_half_off():
    assert apply(100, 50) == 50

def test_rejects_negative():
    with pytest.raises(
        ValueError):
        apply(100, -5)
```

React (Playwright)

```
import { test, expect }
from '@playwright/test';

test('checkout disables', async
({ page }) => {
    await page.goto('/cart');
    const btn = page.getByRole(
        'button', { name: /pay/i });
    await expect(btn).toBeDisabled();
    await page.getByLabel('Terms')
        .check();
    await expect(btn).toBeEnabled();
});
```

Node.js (Jest + supertest)

```
test('GET /users → 200', async () => {
    const res = await request(app)
        .get('/users');
    expect(res.status).toBe(200);
    expect(res.body).toHaveLength(3);
});

test('unknown id → 404', async () => {
    const res = await request(app)
        .get('/users/999');
    expect(res.status).toBe(404);
});
```

Exercise: From the tests alone, describe a one-sentence spec for each function.

Beacons – The Structural Signs Guiding Your Through the Code

Assertions Are Beacons

Zooming in from the file level to the statement level, the individual *assertions* within a test (or embedded within production code) act as highly localized beacons. Check for pre- and post conditions (and invariants) specified in the code.

Tip: Add assertions in your own code to make it easier for others to use top down reading approaches.

Assertions — Code Examples

Pre/Postconditions Pin Down Intent

Python

```
def withdraw(account, amount):  
    # preconditions  
    assert amount > 0  
    assert account.balance >=  
amount  
  
    account.balance -= amount  
  
    # postcondition (invariant)  
    assert account.balance >= 0  
    return account.balance
```

React

```
function useSlider(min, max) {  
    // precondition  
    invariant(min < max);  
    const [v, setV] =  
useState(min);  
    const update = (n) => {  
        invariant(  
            n >= min && n <= max);  
        setV(n);  
    };  
    return [v, update];  
}
```

Node.js

```
const assert = require('assert');  
function transfer(from, to, amt) {  
    // preconditions  
    assert(from !== to);  
    assert(amt > 0);  
    assert(from.balance >= amt);  
    from.balance -= amt;  
    to.balance += amt;  
    // postcondition  
    assert(from.balance >= 0);  
}
```

Exercise: List an input that would crash each function.
The assertions tell you — point to the line.

Beacons – The Structural Signs Guiding Your Through the Code

Diagrams Are Beacons

UML Diagrams help build a top-down mental model of the system.

When reading a code base, first look for diagrams. Visual diagrams often help understanding important complex concepts more efficiently.

However: Sometimes diagrams are **outdated**, which can lead to incorrect hypotheses.

Tips on How to Transition from Bottom-Up Towards Top-Down Code Reading

Continuous Cycle of Hypothesis Testing

Hypothesis Generation: e.g., the developer assumes the system must have a “`database connection`” module.

Beacon Hunting: The developer scans the code looking for beacons, such as an `SQL` library import, a `connectionString` variable, or a `db_connect()` method.

Verification or Rejection: If the anticipated beacons are found, the hypothesis is verified and becomes a part of the mental model; if the beacons are missing, the hypothesis is declined, and the programmer must adjust their assumptions.

Tips on How to Transition from Bottom-Up Towards Top-Down Code Reading

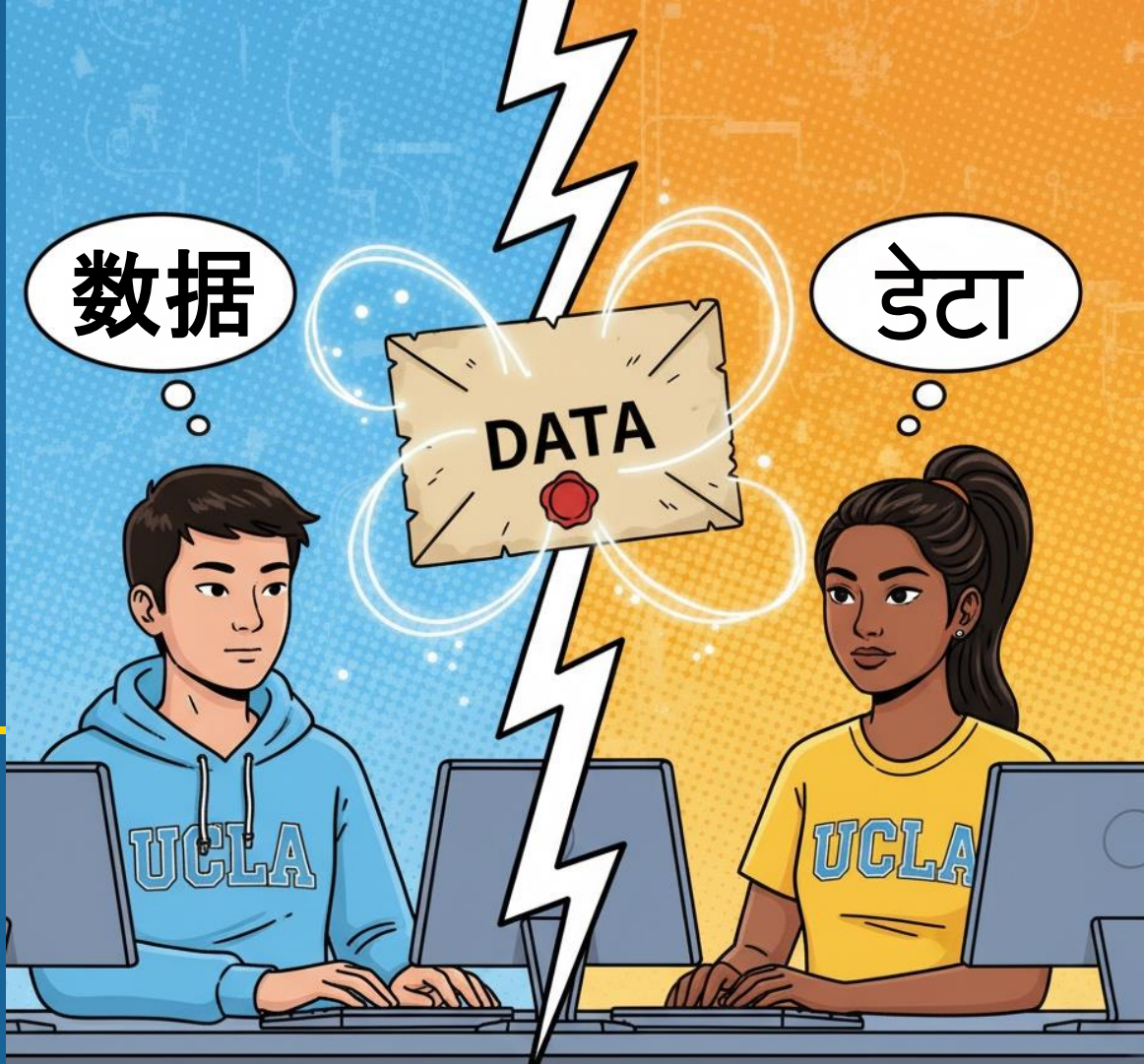
Embrace “Opportunistic Processing”

Experts rarely use top-down processing in strict isolation; instead, they act as "opportunistic processors" who continuously toggle between top-down and bottom-up strategies.

Tip: Use top-down methods to quickly navigate and build the big picture. When you encounter a highly complex, unfamiliar, or localized piece of logic where your hypotheses fail, seamlessly drop into a bottom-up approach to understand that specific chunk, and then immediately return to your top-down overview

API Design & Interoperability

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department



**Bob Wants To Book
Flights So That He
Can Go On Vacation**

**Let me
Tell You
A Story...**



Bob's Flight has been Cancelled



The Airline Agent Tries To find an Alternative Flight

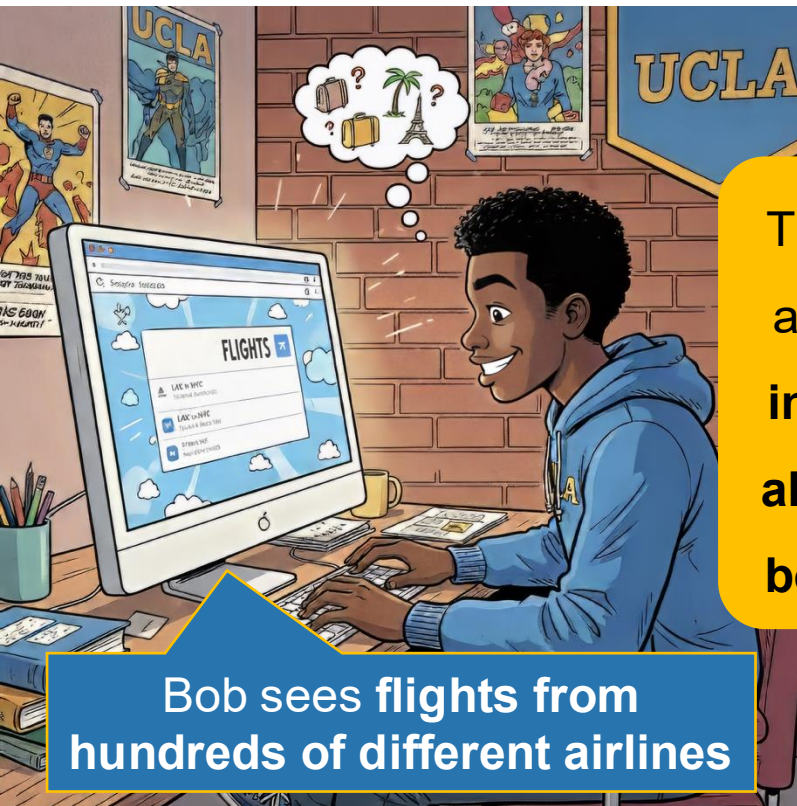


**Bob has been
Re-Booked
Onto a Flight with A
Different Airline**

THE END



What makes this story difficult to implement?



There needs to be a system that can **interoperate with all airlines and all booking systems**

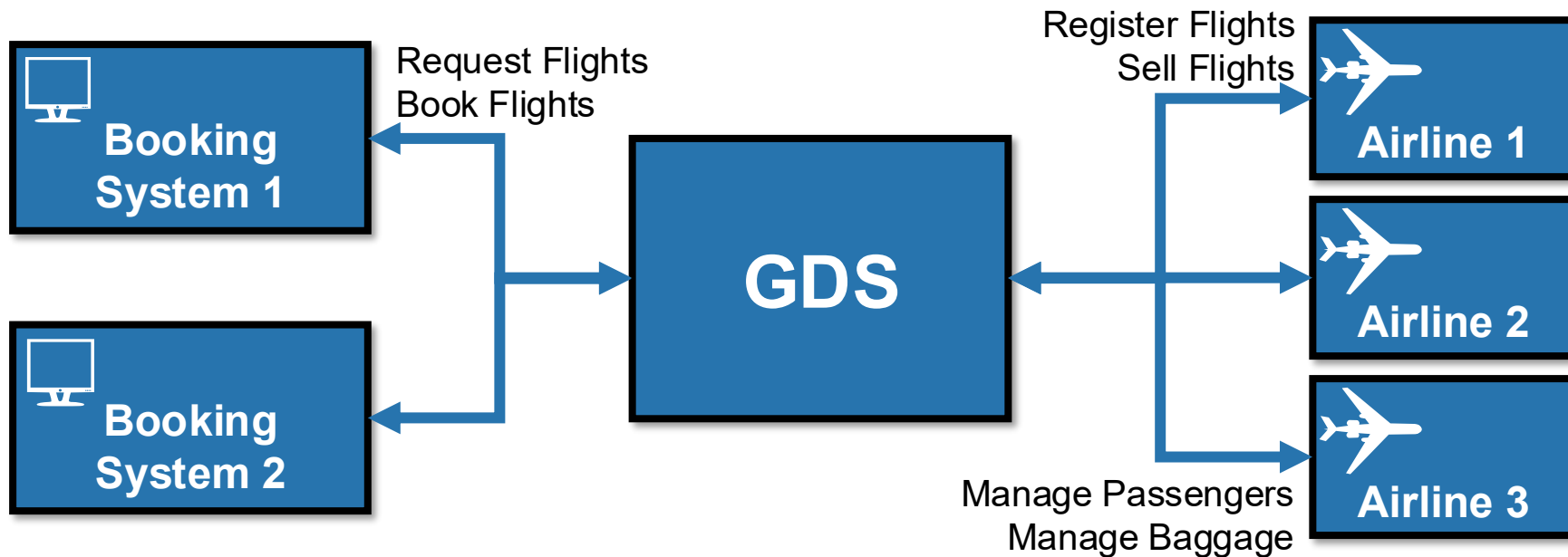
Bob sees **flights from hundreds of different airlines**



Bob can get **re-booked to a different airline**

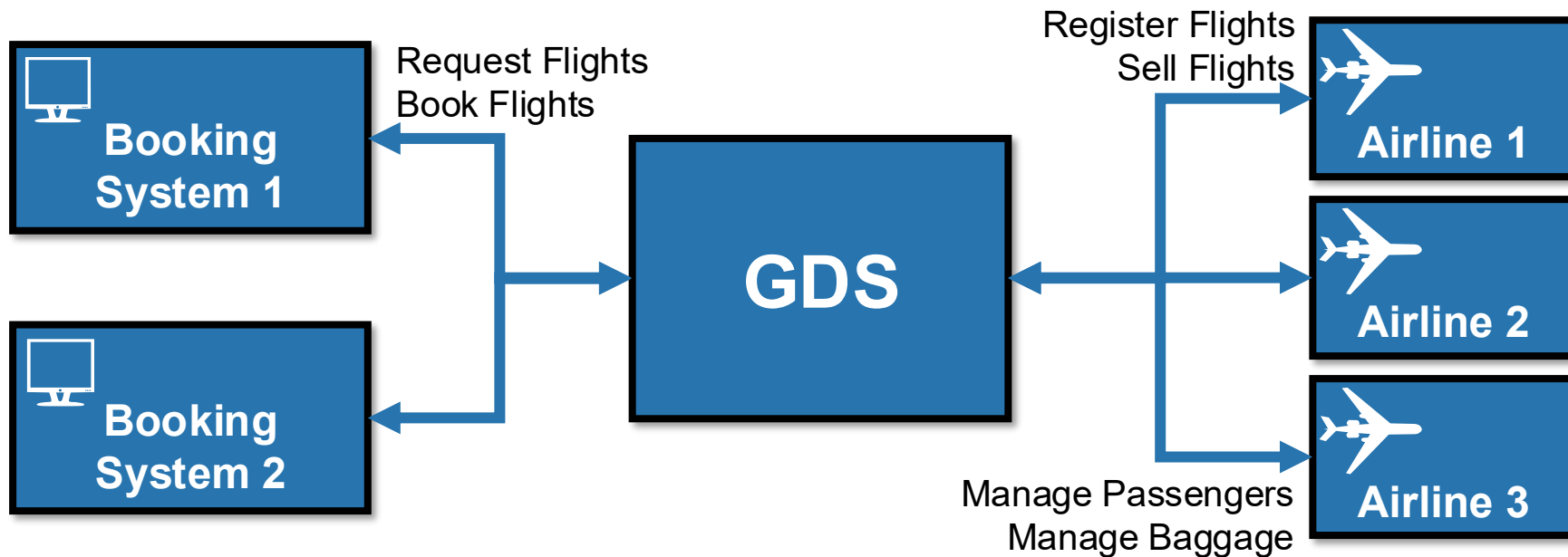
The Global Distribution System (GDS) lets Almost all Airlines & Booking System Interoperate

The invention of GDS is closer historically to the invention of flight by the Wright brothers than to today



What makes GDS Interoperable?

Invent a Design Principle to Design Systems for Interoperability. Talk to Your Neighbor(s)



Design Principle for Interoperability: Create Shared Interfaces / Data Formats



1. **List all data** that needs to be exchanged



2. Define an **interface / data format** that supports all data



3. Implement **serialization & deserialization**

Design Principle for Interoperability: Create Shared Interfaces / Data Formats



Build **Abstractions**

The interface / document format should **not expose any implementation details** of systems / components.



Ensure **Language- and Platform-Independence**

The format should be supported by all programming languages, operating systems, and devices.

Common Technique to Implement Shared Interfaces: REST APIs

- REST (representational state transfer) is a **stateless protocol** (no server-side session) to exchange data in client-server systems via HTTP / HTTPS:
 - **POST** – Creates a Resource
 - **GET** – Reads a Resource
 - **PUT** – Updates a Resource
 - **DELETE** – Deletes a Resource
- Naming convention for URLs, based on resource identifiers
 - “/users/123” instead of e.g., many variations of /getUser?id=123
- Resources are often described via XML, YAML, JSON, or HTML

Example REST API: GDS API

- Request: `POST /getFlights`

```
...  
"travelPreferences": {  
    "flightType": "Direct",  
    "maximumStopsQuantity": 1  
},  
"itineraryParts": [ {  
    "departureAirportCode": "LAX",  
    "arrivalAirportCode": "PIT",  
    ... ] }  
...
```

RESPONSES

Response content type

CODE

DESCRIPTION

200

Successful response or application layer errors held in the response.

Example Value

Model

```
"locationCode": "DFW",  
"arrivalDateTime": "2019-02-20T05:50:00",  
"departureDateTime": "2019-02-20T05:50:00",  
"elapsedTime": 40,  
"duration": 40,  
"gmtOffset": 11,  
"equipmentType": "73H"
```

Alternative Techniques to Implement Shared Interfaces (besides REST)

RPC

(Remote Procedure Call)

Calling a function on a remote server as if it were local. Can be stateful. **Functions** and **actions** beyond CRUD. Good for **complex calculations**. Can use multiple document formats.

SOAP

(Simple Object Access Protocol)

Can be stateful. More **complex**. Sometimes **slower**. Has more **security** features. Has built-in **error handling**. Good for **distributed enterprise environments**. Uses XML.

GraphQL

Server-side schema defines types, enabling checking of data structure conformance. Good for **large, complex**, and **interrelated** data sources. Uses JSON.

Common Technique to Test Compatibility: XML / JSON / YAML Schema

- A schema **describes the structure** of document
- Schemas list attributes, their possible values, and complex types (e.g., sequences, recursive types) for a document
- **Validation** of a document against a schema can be done **automatically** to **test compatibility at run-time**

Example JSON Schema

```
"properties": {  
  "departureAirportCode": {  
    "type": "string"  
    "pattern": "^[A-Z]{3}$"  
  },  
  "price": {  
    "type": "number",  
    "minimum": 0,  
    "exclusiveMinimum": true  
  }  
},  
"required": ["departureAirportCode", "price"]
```

Constraints on the Type of a Property

RegEx!

Constraints on the Values of a Property

Constraints on Document Structure

Case Study

Mars Climate Orbiter

Question: How could they have prevented this bug?

**Spacecraft Lost Due to Lack of
Semantic Interoperability
(Shared Interpretation of Shared Data)**

Flight System Software

Developed by NASA JPL

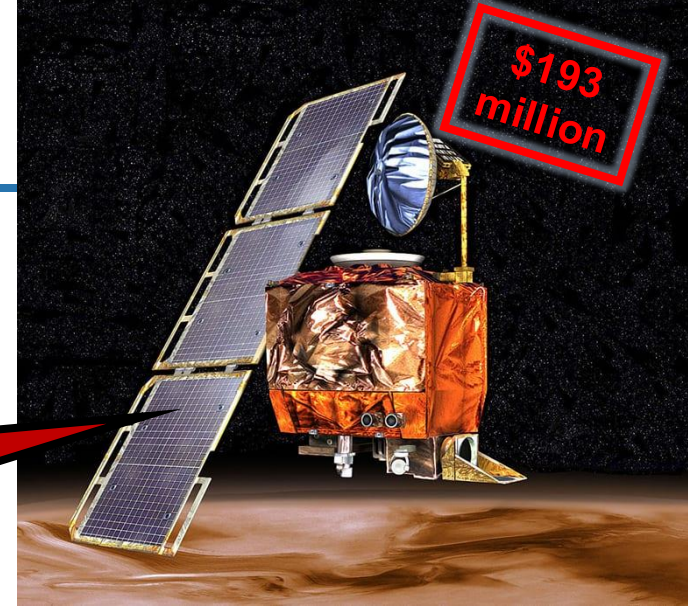
Expected commands in
N (SI units)

**Command
Interface**

Ground Software

Supplied by Lockheed Martin
(US-based sub-contractor)

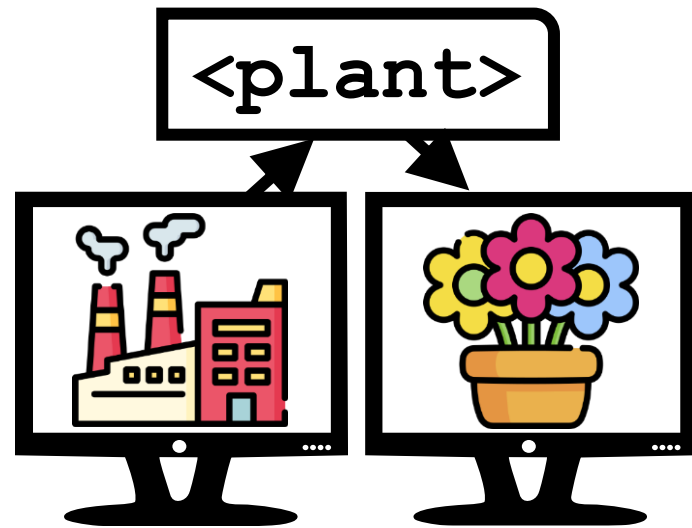
Sent commands in
lbf (US Customary units)



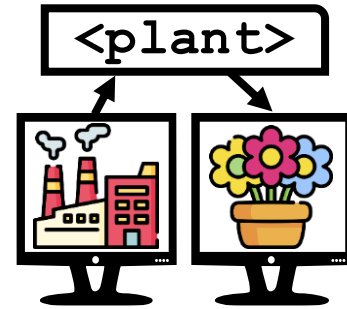
Lesson Learned: Syntactic Interoperability is Not Enough

In-Class Activity: Invent a Design Principle for Semantic Interoperability

Data exchanged between systems / components must be **interpreted** in the same way by all systems / components.



Design Principle for Interoperability: Define the Semantics of Shared Data



- **Document Interfaces** and their **Semantics**

(e.g., What units are implied? Does price include tax?

MM/DD/YY or DD/MM/YY?

What coordinate system is used a reference frame?)

- **Use Shared Dictionary of Items** or **Agree on Vocabulary**

(e.g., doughnut or donut? 单丛茶 or 单枞茶?)

- Write **Integration Tests** for the Systems

Interface Descriptions

Syntactic View

Describe document format, the actions that can be performed, their parameters, and outputs.

Semantic View

Describe the purpose / meaning of the resource / action:

- **Side-effects:** Changes to the state of a resource or environment
- **Usage restrictions:** Who can perform this action?
- **Error Handling:** What errors can occur and why?
- **Examples:** Examples of outputs for a given input

(Simplified) Syntactic View of the GDS Flight Booking API

POST /createBooking {

- flightNumber: str

The flight number using the code of the operating airline

- date: dateTime

The date of the departing flight according to the time zone of the departure airport

- seat: str

Seat number (same label as on the plane)

- paymentReference: PaymentMethod

- travelerName: str

Full name of the person traveling in ACSII

}

In-Class Activity: Describe The Semantic View of GDS Flight Booking API

Example: Semantic View of GDS Booking

- **Purpose:** The airline confirms a requested booking
- **Side-effects:** money is charged, seat is marked as sold, can be canceled within 1 hour
- **Usage restrictions:** Authorized booking systems
- **Errors:** Invalid Format, Unauthorized, too many requests, ...

See here: https://developer.sabre.com/docs/rest_apis/trip/orders/booking_management

Guidelines on Interface Documentation

- Focus on how **elements interact** and their **externally visible effects**, not their implementation
- To support **changeability** of the implementation, expose only **what is needed** to use the interface
- Keep the documentation **minimal** and **use-case oriented** to increase **readability**

GDS Interface Documentation

POST

/getFlights

In-Class Activity: Describe Pros and Cons of the GDS Interface Documentation

Creates a list of itineraries with the given search criteria during exchange. The response contains a list of itineraries with fare offerings and detailed price breakdown.

PARAMETERS

CANCEL

NAME	DESCRIPTION
X-Request-ID string (header)	The unique ID of the request to track flow level transactions. X-Request-ID
AirSearchRequest t object	Contains the required parameters to search for the itineraries. Edit Value Model

https://developer.sabre.com/docs/rest_api/trip/orders/booking_management

GDS Interface Documentation

In-Class Activity: Describe Pros and Cons of the GDS Interface Documentation

**Semantic
Interoperability
Via Shared Dictionary**

OnTimePerformance ∨ {

description:

Contains the statistical data
how often the flight is on time.

level

string
pattern: `^[0-9]{1}$`
example: 1

The numeric value (0-9) associated
with the level of on time
performance of the flight.

AirportCode ∨ string

pattern: `^[A-Z]{3}$`

example: DFW

The three-letter IATA code of the airport.

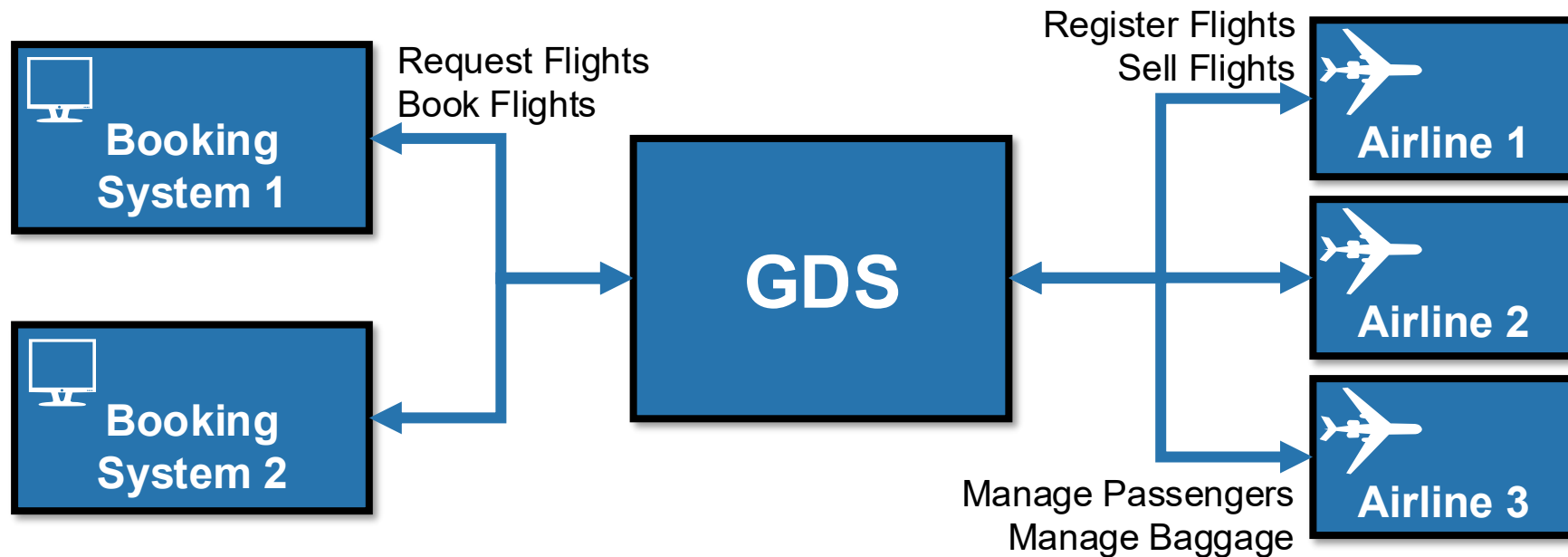
Semantics of Level Are Unclear

However ... Now Many Airlines are Moving Away From GDS

- GDS only lets you book seats
- Airlines make more money **up-selling “priority boarding”**, or other **add-ons**
- Southwest **doesn’t have assigned seating**
- Some airlines **have seats that can be transformed** into a sky couch, resulting in the unavailability in GDS
- Airlines do not get valuable **customer data** via GDS

What makes GDS **Less Changeable**?

In-Class Activity: Generate Design Options to Increase the Changeability of GDS Booking Options



Making GDS More Changeable

Extensible Interfaces:

- Offers can contain a dynamically-defined add-ons
- Add-on: (`price`, `name`, `description`, `id`)
 - `price(int)`: The price in cents (excl. tax) additionally charged when this add-on is selected
 - `name(str)`: The name of the add-on as shown to the user (in UTF-8)
 - `description(str)`: A short description shown to the user in order to decide if they want to purchase the add-on (in UTF-8)
 - `id(str)`: Unique identifier of this add-on starting with the flight number (in ASCII)
- A list of add-ons is added to a flight listing. The booking API needs to add an optional list of add-on `ids` to identify requested add-ons.

Harder to Implement

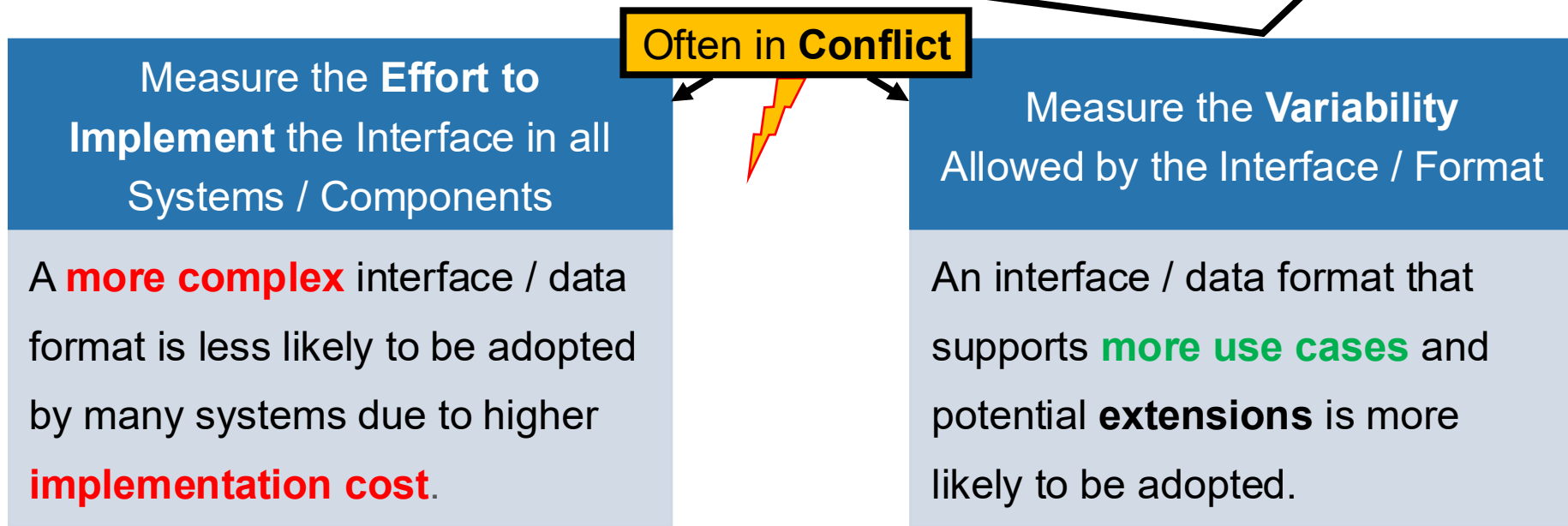
In-Class Activity: What Disadvantage does this have over the existing GDS?

Lesson Learned: Interoperability Often Conflicts with Changeability

- Shared interfaces and data format mean changes have to be implemented **in all** cooperating systems
- We **cannot localize** the interface change to a single system, so extensions / changes require a new version of the interface, which breaks interoperability

How to Evaluate Practical Interoperability?

These **Design-Time Measures** evaluate how likely a **design** for interoperability is to succeed in a practical setting. They can be used in quality attribute specifications.



Design Pattern for Interoperability: Use *Adapters* to Connect Interfaces

Question: Does
this only work
for Syntactic
Interoperability?

Problem: Two systems use **different interfaces**
(e.g., different data formats, different protocols, ...)



Solution: Create an **adapter component** that **translates** between the two interfaces.



Please Fill out your Exit Tickets on Bruin Learn!

Question 1

1 pts

In your own words, please summarize **three key insights** you learned about **Code Comprehension** today. (Do not copy phrases from the lecture slides)

Question 2

1 pts

For one API that is used in your project for client server communication, write a first draft of an **interface specification**. Include **syntax** (parameters and their type) and **semantics** (what is the meaning of the overall request & the parameters? Which Errors could occur, ...)

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slides use images generated with Gemini and from Flaticon.com (Creators: Freepik)